

综合实验二：基于晶体管的温度传感器

严子竣，朱浩男，于耀博，郭健

2026 年 7 月 13 日

1 引言

本实验利用双极型晶体管基极-发射极电压差 (ΔV_{BE}) 与绝对温度成正比的物理特性 (PTAT 原理)，构建一种高精度温度传感器。通过电流镜产生固定电流比，获取微弱的 PTAT 电压信号，再经差分放大和电平移位，将信号调理至适合 ADC 采集的 0~5V 范围，最终实现环境温度的数字显示。

2 实验目的

- 建立传感器概念，培养综合应用电子电路的能力；
- 学习小型电子系统的设计、安装和调试方法；
- 培养工程实践中分析问题、解决问题的能力。

3 电路设计

整个系统分为两大部分：测温前端 (PTAT 电流镜) 和信号调理 (差分放大 + 电平移位)。如图 1 所示。

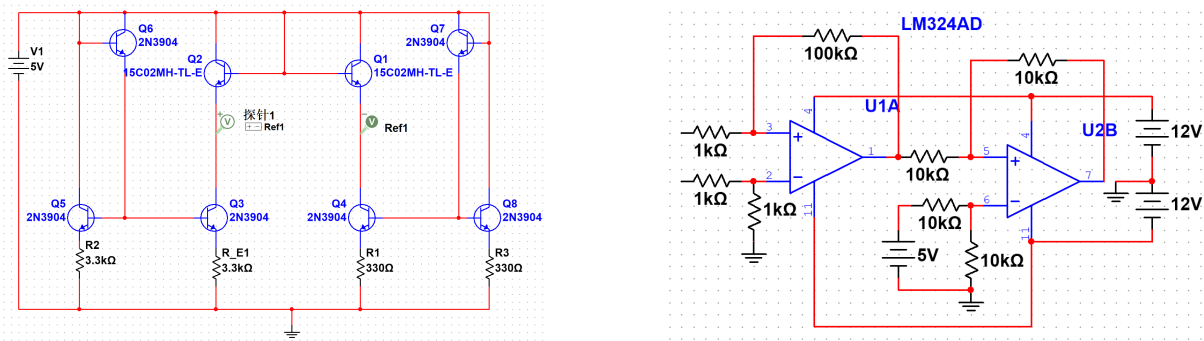


图 1: 左：带 β -helper 的电流镜（电流比约 10:1）；右：LM324 实现的差分放大与电平移位电路

左侧电路采用带 β -helper 的改进型电流镜，通过发射极电阻 ($R_1 = 330\Omega$ 与 $R_2 = 3.3k\Omega$) 设置两臂电流比约为 10:1，从而在两只对管 (Q1/Q2) 的基-射极之间产生 PTAT 电压 V_{PTAT} 。右侧电路第一级为差分放大器 (增益约 100 倍)，第二级为电平移位器，将输出的共模电平降低约 5V，使其适配 Arduino 的 0~5V ADC 输入。

4 理论计算

4.1 实测器件参数

实际测得电流镜左右臂电阻上的电压分别为：

- 右侧臂 ($R = 327.8\Omega$): $V_R = 3.538V$;
- 左侧臂 ($R = 3.271k\Omega$): $V_L = 3.660V$ 。

由此计算两路集电极电流：

$$I_1 = \frac{3.660}{3.271 \times 10^3} = 1.119 \times 10^{-3} A \quad (1)$$

$$I_2 = \frac{3.538}{327.8} = 10.793 \times 10^{-3} A \quad (2)$$

电流比

$$n = \frac{I_2}{I_1} = \frac{10.793}{1.119} \approx 9.646 \quad (3)$$

与设计值 10 吻合良好。

4.2 PTAT 电压温度系数

根据半导体物理，PTAT 电压满足

$$V_{PTAT} = \frac{kT}{q} \ln n \quad (4)$$

其对温度求导得

$$\frac{dV_{PTAT}}{dT} = \frac{k}{q} \ln n \quad (5)$$

其中 $\frac{k}{q} = 86.17 \mu V/K$ 。代入 $n = 9.646$ ：

$$\frac{dV_{PTAT}}{dT} = 86.17 \times 10^{-6} \times \ln(9.646) = 195.49 \mu V/K \quad (6)$$

该值即为 PTAT 电压的理论温度灵敏度。

4.3 放大后分辨率

第一级差分放大增益实测 $A_v = 100.65$ ，则放大后的输出电压温度灵敏度为

$$\frac{dV_{\text{out}}}{dT} = A_v \cdot \frac{dV_{\text{PTAT}}}{dT} = 100.65 \times 195.49 \mu\text{V/K} = 19.67 \text{ mV/K} \quad (7)$$

即每开尔文（摄氏度）温度变化对应 19.67mV 的输出电压变化，这是系统的理论分辨率。

5 实际校准与测量范围

使用商用数字温度传感器（DS18B20）作为参考，对设计系统进行两点校准。记录数据如下：

温度（℃）	输出模拟电压（V）	备注
27.1	1.271	室温
110.0	2.928	热风枪加热

由两点计算实际灵敏度：

$$\frac{\Delta V}{\Delta T} = \frac{2.928 - 1.271}{110.0 - 27.1} = \frac{1.657}{82.9} = 19.98 \text{ mV/}^\circ\text{C} \quad (8)$$

该值与理论值 19.67mV/K 高度吻合，验证了设计的正确性。

根据校准直线（ $V = 19.98 \times 10^{-3}T + b$ ），由 $T = 27.1$ 时 $V = 1.271$ 求得截距 $b = 1.271 - 19.98 \times 10^{-3} \times 27.1 = 0.730\text{V}$ ，即

$$V = 0.01998 T + 0.730 \quad (9)$$

反解温度：

$$T = \frac{V - 0.730}{0.01998} \quad (10)$$

Arduino ADC 输入范围为 0~5V，故可测量的温度范围为：

$$T_{\min} = \frac{0 - 0.730}{0.01998} = -36.5^\circ\text{C} \quad (11)$$

$$T_{\max} = \frac{5 - 0.730}{0.01998} = 213.7^\circ\text{C} \quad (12)$$

实际由于运放供电为 $\pm 12\text{V}$ ，电平移位后最高输出可达 5.5V，但受 ADC 量程限制，有效范围仍为 0~5V，最终测量范围为 $-36.5^\circ\text{C} \sim 213.7^\circ\text{C}$ ，可根据实际需求略微下调运放供电电压、增大差分放大倍数以进一步改进。

6 电平移位的作用

若不进行电平移位，差分放大器输出为双极性或共模电压较高，无法直接接入单极性 ADC。通过第二级运放构建的电平移位电路：

- 将输出电压整体下移约 5V，使其落在 0~5V 区间；
- 充分利用 ADC 的满量程范围，避免因信号幅度过小导致量化误差占主导；
- 使得温度分辨率由简单的 8 位提升至接近 10 位（实际使用 10 位 ADC），相比不移位方案分辨率提升约一倍。

本设计中，电平移位后的灵敏度约为 $20\text{mV}/^{\circ}\text{C}$ ，使得每 $^{\circ}\text{C}$ 对应约 4 个 ADC 码值 ($5\text{V}/1024 \approx 4.88\text{mV}$)，可分辨 0.25°C 的温度变化，满足精度要求。

7 实验现象

室温下测温电路与商用温度传感器相比，温度差距基本在 1°C 以内；使用热风枪加热时，温度很快上升至 100°C 以上，相比商用温度传感器温度上升较快，可能是由于商用温度传感器不考虑温度剧烈变化的情况，使用了较大的电容进行稳压。在热风枪下工作一段时间后，两温度计的温度都稳定在 106°C 左右，且冷却后温度计仍与商用温度计相差 1°C 以内，说明测温电路能够较为准确的测量温度。

8 总结

本文基于 PTAT 原理设计并实现了一款温度传感器，主要成果如下：

1. 采用带 β -helper 的电流镜产生稳定的 10:1 电流比；
2. 通过 100.65 倍差分放大和电平移位，将灵敏度提升至 $19.67\text{mV}/\text{K}$ ，实际校准灵敏度为 $19.98\text{mV}/\text{K}$ ，误差小于 2%；
3. 理论有效测量范围覆盖 -36.5°C 至 213.7°C ，远超基本要求；
4. 电平移位策略显著提高了 ADC 量程利用率，使得系统分辨率较简单放大方案提升一倍。

实验表明，该方案具有成本低、线性度好、易于与数字系统接口等优点，具有较高的工程实用价值。

A Arduino 数据采集与显示代码

以下为 Arduino 端完整程序，实现模拟传感器（A0）与数字参考传感器（DS18B20）的双通道采集，并经线性校准后显示于 OLED 屏幕。

Listing 1: Arduino 测温程序

```
#include <U8g2lib.h>
#include <Wire.h>
#include <OneWire.h>
#include <DallasTemperature.h>

// OLED 初始化 (硬件 I2C)
UG2_SSD1306_128X64_NONAME_1_HW_I2C u8g2(U8G2_R0,
    /* reset=*/ U8X8_PIN_NONE);

// ----- DS18B20 配置 -----
#define ONE_WIRE_BUS 8           // DQ 接 Arduino D2
OneWire oneWire(ONE_WIRE_BUS);
DallasTemperature sensors(&oneWire);

// ----- 温度标定参数 (根据实际电路调整) -----
const float V_LOW    = 1.271;    // 低温校准点电压
const float Temp_LOW = 27.1;     // 低温温度
const float V_HIGH   = 2.928;    // 高温校准点电压
const float Temp_HIGH= 110;      // 高温温度
// -----

void setup() {
    u8g2.begin();
    sensors.begin();              // 初始化 DS18B20
}

void loop() {
    // ----- 读取 A0 模拟传感器 (软件滤波) -----
    const int NUM_SAMPLES = 20;
    long sum = 0;
    for (int i = 0; i < NUM_SAMPLES; i++) {
        sum += analogRead(A0);
```

```

}
int raw = sum / NUM_SAMPLES;

// 转换为电压 (假设 Arduino 工作电压 5V)
float voltage = raw * (5.0 / 1023.0);

// 线性插值计算 T1 (模拟传感器温度)
float SCALE = (V_HIGH - V_LOW) / (Temp_HIGH - Temp_LOW);
float T1 = (voltage - V_LOW) / SCALE + Temp_LOW;
// -----

// ----- 读取 DS18B20 温度 (T2) -----
sensors.requestTemperatures();
float T2 = sensors.getTempCByIndex(0);
if (T2 == DEVICE_DISCONNECTED_C) {
    T2 = -999.0;    // 传感器未连接时的错误标记
}
// -----

// ----- OLED 显示 -----
u8g2.firstPage();
do {
    // 第一行: T1 (大字体)
    u8g2.setFont(u8g2_font_ncenB14_tr);
    u8g2.setCursor(0, 25);
    u8g2.print("T1:");
    u8g2.print(T1, 1);
    u8g2.print("C");

    // 第二行: T2 (小字体)
    u8g2.setFont(u8g2_font_6x10_tr);
    u8g2.setCursor(0, 45);
    u8g2.print("T2:");
    if (T2 == -999.0) {
        u8g2.print("Error");
    } else {
        u8g2.print(T2, 1);
    }
}

```

```

        u8g2.print("C");
    }

    // 第三行：A0 电压（小字体）
    u8g2.setCursor(0, 58);
    u8g2.print("A0:");
    u8g2.print(voltage, 3);
    u8g2.print("V");

} while (u8g2.nextPage());
// -----

delay(1000);    // 1秒刷新一次
}

```

代码要点说明：

- 采用 20 次均值滤波抑制随机噪声；
- 基于两点标定参数进行线性插值，将采集电压转换为温度；
- 同时读取 DS18B20 作为参考，用于实时比对校准；
- OLED 分三行显示模拟温度、参考温度和原始电压，刷新闻隔 1 秒。